Generic Methods

CS300: Programming II

Cole Nelson

Learning Objectives

1. Be able to **trace** how `compareTo` orders a list.
2. Be able to **write** a generic method with a type parameter `<T>`.
3. Be able to **identify** the methods an unbounded `T` can call.

Recap: `compareTo` answers with a number

```
a.compareTo(b) < 0      // a goes first
a.compareTo(b) == 0     // a tie
a.compareTo(b) > 0      // b goes first
```

Not a `boolean`. Only the sign matters.

Song implements Comparable<Song>

By length? By title? **We decide.**

```
public class Song extends Track implements Comparable<Song> {
  @Override
  public int compareTo(Song other) {
    return this.getSeconds() - other.getSeconds();   // shortest first
  }
}
```

Song implements Comparable<Song>

By length? By title? **We decide.**

```
public class Song extends Track implements Comparable<Song> {
  @Override
  public int compareTo(Song other) {
    return this.getTitle().compareTo(other.getTitle());   // A to Z
  }
}
```

Eclipse Exercise

Observation of interfaces and `compareTo`.

Tophat Q1

Song's `compareTo` is now:

```
return other.getSeconds() - this.getSeconds();
```

Jump Around is 217 seconds, and Shake It Off is 219.
What does `jumpAround.compareTo(shakeItOff)` return?

→ Open Tophat to vote. *Then* we run it in Eclipse.

We've been filling in types all along

```
ArrayList<Song> library = new ArrayList<Song>();
ArrayList<String> artists = new ArrayList<String>();

public class Song extends Track implements Comparable<Song>
```

`ArrayList` and `Comparable` were each written **once**, with the type left blank. That's **generics**.

Same method, twice

```java
public static ArrayList<Song> firstNSongs(ArrayList<Song> list, int n) {
  ArrayList<Song> result = new ArrayList<Song>();
  for (int i = 0; i < n; i++) {
    result.add(list.get(i));
  }
  return result;
}

public static ArrayList<String> firstNStrings(ArrayList<String> list, int n) {
  ArrayList<String> result = new ArrayList<String>();
  ...   // the same loop, and the same return
}
```

Only the type changed. A list of podcasts would need a third, a list of playlists a fourth, etc. **Not ideal!**

One method for everything: `Object` ?

```
public static ArrayList<Object> firstNItems(ArrayList<Object> list, int n) {
  ArrayList<Object> result = new ArrayList<Object>();
  for (int i = 0; i < n; i++) {
    result.add(list.get(i));
  }
  return result;
}
```

Every class is an `Object` . So this takes anything... right?

Pass in our songs?

```
ArrayList<Song> library = new ArrayList<Song>();
library.add(brightside);

firstNItems(library, 3);   // not applicable for the arguments (ArrayList<Song>, int)
```

Is an `ArrayList<Song>` an `ArrayList<Object>`? **No!**

Not allowed by Java.

While a `Song` is an `Object`, an `ArrayList<Song>` is **not** an `ArrayList<Object>`.

A generic method

```
public static <T> ArrayList<T> firstNItems(ArrayList<T> list, int n) {
  ArrayList<T> result = new ArrayList<T>();
  for (int i = 0; i < n; i++) {
    result.add(list.get(i));
  }
  return result;
}
```

`<T>` goes **before** the return type. `T` holds a *type*, the way a parameter holds a value.

Java infers `T` from the passed argument

```
firstNItems(library, 3)    // T is Song: an ArrayList<Song> comes back
firstNItems(artists, 2)    // T is String
firstNItems(episodes, 2)   // T is Podcast
```

`T` is always a class: `ArrayList<Integer>`, never `ArrayList<int>`.

Eclipse Exercise

Get today's starter from Canvas.

TODO 1. One `firstNItems` for songs, artists, *and* episodes.

Why won't this compile?

```
public static <T> int totalSeconds(ArrayList<T> tracks) {
  int total = 0;
  for (T t : tracks) {
    total += t.getSeconds();
  }
  return total;
}
```

We only ever pass it songs and podcasts...

A `T` could be anything

```
t.toString();        // fine: every object has one
t.equals(other);     // fine
t.getSeconds();      // The method getSeconds() is undefined for the type T
```

`T` could be a `String`, and a `String` has no `getSeconds()`. So a plain `T` only gets what every object has: **`Object`'s methods.**

Can we promise `T` is a `Track`? **Yes!**

Bounds

```
public static <T extends Track> int totalSeconds(ArrayList<T> tracks) {
  int total = 0;
  for (T t : tracks) {
    total += t.getSeconds();   // compiles: every Track has one
  }
  return total;
}
```

`T extends Track`: `T` **must be a** `Track`**,**
or a kind of `Track`**.**

Bounds

```
public static <T extends Track> int totalSeconds(ArrayList<T> tracks)

ArrayList<Song> library = new ArrayList<Song>();
ArrayList<String> artists = new ArrayList<String>();

totalSeconds(library);   // T is Song, a kind of Track
totalSeconds(artists);   // not applicable for the arguments (ArrayList<String>)
```

`T extends Track`: `T` **must be a** `Track`**,**
or a kind of `Track`**.**

Tophat Q2

Inside this method, which line compiles?

```
public static <T> void show(T item)
```

→ Open Tophat to vote.

Questions?